

Prueba técnica

Gestión de un Cuadrangular de Fútbol

Documentación Técnica del Proyecto

Luisa Fernanda Velasco Lopez

19 de junio de 2026

Oficina Asesora de Tecnologías e Información de la Universidad
Distrital Francisco José de Caldas.

Índice

1. Introducción	4
2. Planteamiento del Problema	4
3. Objetivos	4
3.1. Objetivo General	4
3.2. Objetivos Específicos	5
4. Cumplimiento del Enunciado	5
5. Repositorio del Proyecto	6
6. Acceso a la Aplicación	6
7. Stack Tecnológico	6
8. Arquitectura de la Solución	7
9. Estructura del Proyecto	7

10.Modelo de Datos	9
10.1. Diagrama Entidad-Relación	9
10.2. Modelo Relacional	9
11.Diccionario de Datos	10
11.1. Entidad Usuario	10
11.2. Entidad Equipo	10
11.3. Entidad Partido	10
12.Reglas de Negocio	10
13.Implementación de la API REST	11
13.1. Autenticación	11
13.2. Equipos	11
13.3. Partidos	11
13.4. Tabla de posiciones	12
13.5. Reinicio del torneo	12
14.Buenas Prácticas de Desarrollo	12
14.1. Arquitectura Modular	12
14.2. Separación de Responsabilidades	12
14.3. Validación de Datos	12
14.4. Manejo de Excepciones	13
15.Autenticación y Seguridad	13
16.Persistencia de Datos	14
17.Contenedorización con Docker	14
18.Despliegue en Producción	14
19.Control de Versiones	15
20.Pruebas Automatizadas	15
21.Documentación Automática	16
22.Evidencias de Funcionamiento	17
22.1. Inicio de Sesión	17
22.2. Administración de Equipos	18
22.3. Gestión de Marcadores	19

22.4. Tabla de Posiciones	20
22.5. Documentación Swagger	20
22.6. Repositorio GitHub	21
23. Conclusiones	21

1. Introducción

El presente documento describe el diseño, desarrollo e implementación de una API REST para la gestión de un cuadrangular de fútbol. La solución fue construida utilizando una arquitectura moderna basada en servicios, con persistencia en base de datos relacional, autenticación mediante JSON Web Tokens (JWT), despliegue en la nube y una interfaz web que permite interactuar con todas las funcionalidades del sistema.

El proyecto responde al enunciado propuesto, el cual requiere registrar hasta cuatro equipos participantes, generar automáticamente un calendario bajo la modalidad de todos contra todos, almacenar los resultados de los encuentros y calcular dinámicamente la tabla de posiciones.

Además de satisfacer los requisitos funcionales, la implementación incorpora buenas prácticas de ingeniería de software, separación por capas, documentación automática mediante Swagger, integración continua, contenedorización con Docker y despliegue en Microsoft Azure.

2. Planteamiento del Problema

Se requiere desarrollar una aplicación que permita administrar un torneo de fútbol conformado por un máximo de cuatro equipos.

El sistema debe garantizar la correcta organización de un cuadrangular bajo el formato de todos contra todos, donde cada equipo se enfrenta exactamente una vez contra los demás participantes. Asimismo, debe permitir registrar y modificar los resultados de los encuentros disputados y calcular automáticamente la clasificación final del torneo.

La solución debe exponerse mediante una API REST, mantener la persistencia de la información utilizando una base de datos y, preferiblemente, ofrecer una interfaz web para facilitar la interacción con los usuarios.

3. Objetivos

3.1. Objetivo General

Diseñar e implementar una API REST que permita administrar un cuadrangular de fútbol mediante el registro de equipos, generación automática del fixture, gestión de marcadores y cálculo dinámico de la tabla de posiciones.

3.2. Objetivos Específicos

- Registrar hasta cuatro equipos participantes.
- Garantizar la generación automática del calendario bajo el formato todos contra todos.
- Registrar y actualizar los marcadores correspondientes a cada partido.
- Calcular automáticamente la tabla de posiciones considerando puntos, diferencia de gol y goles a favor.
- Persistir toda la información en una base de datos PostgreSQL.
- Implementar autenticación segura utilizando JSON Web Tokens.
- Exponer toda la funcionalidad mediante una API REST documentada automáticamente con OpenAPI (Swagger).
- Proporcionar una interfaz web amigable para facilitar la administración del torneo.

4. Cumplimiento del Enunciado

La solución desarrollada satisface completamente los requerimientos planteados en el enunciado de la prueba técnica.

Requisito	Cumplimiento
Registro de hasta cuatro equipos	Sí
Generación automática del todos contra todos	Sí
Consulta y modificación de marcadores	Sí
Cálculo automático de tabla de posiciones	Sí
Persistencia de datos	Sí
API REST	Sí
Separación entre lógica de negocio y presentación	Sí
Uso de framework moderno (FastAPI)	Sí
Control de versiones con GitHub	Sí
Documentación técnica	Sí
Swagger/OpenAPI	Sí
Autenticación mediante JWT	Sí
Uso de Docker	Sí
Despliegue en la nube	Sí
Interfaz web funcional	Sí
Pruebas automatizadas	Sí

Cuadro 1: Verificación de cumplimiento de los requisitos planteados.

5. Repositorio del Proyecto

El código fuente se encuentra versionado utilizando Git y alojado públicamente en GitHub.

Repositorio oficial:

<https://github.com/Milu2662/Futbol-api>

El uso de control de versiones permitió mantener un historial organizado del desarrollo, implementar una estrategia basada en ramas y facilitar la integración continua mediante GitHub Actions.

6. Acceso a la Aplicación

La aplicación se encuentra desplegada sobre Microsoft Azure y puede accederse directamente desde cualquier navegador web.

Interfaz Web

<https://futbol-api-milu2662.azurewebsites.net/app/>

Documentación Swagger

<https://futbol-api-milu2662.azurewebsites.net/docs>

Credenciales de acceso

Para facilitar la evaluación del proyecto se proporciona un usuario administrador pre-configurado.

Usuario	admin
Contraseña	Admin2026

Cuadro 2: Credenciales de acceso al sistema.

7. Stack Tecnológico

La Tabla 3 resume las tecnologías empleadas durante el desarrollo del proyecto.

Componente	Tecnología
Lenguaje	Python 3.13
Framework Backend	FastAPI
Servidor ASGI	Uvicorn
ORM	SQLAlchemy 2.0
Migraciones	Alembic
Base de datos	PostgreSQL 16
Validación	Pydantic v2
Autenticación	JWT
Frontend	HTML, CSS y JavaScript
Pruebas	Pytest
Contenedores	Docker y Docker Compose
Integración Continua	GitHub Actions
Despliegue	Microsoft Azure
Repositorio de imágenes	Docker Hub

Cuadro 3: Stack tecnológico empleado en el proyecto.

8. Arquitectura de la Solución

La aplicación fue desarrollada siguiendo una arquitectura en capas con el propósito de mantener una adecuada separación de responsabilidades y facilitar la escalabilidad y el mantenimiento del código.

Cada componente cumple una función específica dentro del sistema:

- **Frontend:** interfaz desarrollada en HTML, CSS y JavaScript que consume la API REST.
- **API REST:** implementada con FastAPI, expone los servicios mediante endpoints HTTP.
- **Capa de servicios:** contiene la lógica de negocio del cuadrangular.
- **Modelos:** representan las entidades persistidas en PostgreSQL utilizando SQLAlchemy.
- **Base de datos:** almacena permanentemente la información de usuarios, equipos y partidos.

9. Estructura del Proyecto

El proyecto se encuentra organizado siguiendo una estructura modular que facilita la mantenibilidad y separación de responsabilidades.

Futbol-api/

backend/

app/

api/

core/

db/

models/

schemas/

services/

main.py

alembic/

tests/

Dockerfile

frontend/

css/

js/

index.html

docs/

docker-compose.yml

README.md

Cada directorio posee una responsabilidad claramente definida:

- **api**: definición de endpoints REST.
- **services**: implementación de reglas de negocio.
- **models**: definición de entidades persistentes.
- **schemas**: validación de datos mediante Pydantic.
- **db**: conexión con PostgreSQL.
- **frontend**: interfaz web consumidora de la API.
- **tests**: pruebas automatizadas del sistema.

10. Modelo de Datos

El sistema utiliza una base de datos relacional PostgreSQL compuesta por las entidades Usuario, Equipo y Partido. La información se encuentra normalizada y modelada para garantizar la integridad referencial y facilitar el cálculo dinámico de la tabla de posiciones.

10.1. Diagrama Entidad-Relación

La Figura 1 presenta el diagrama entidad-relación utilizado en el desarrollo del proyecto.

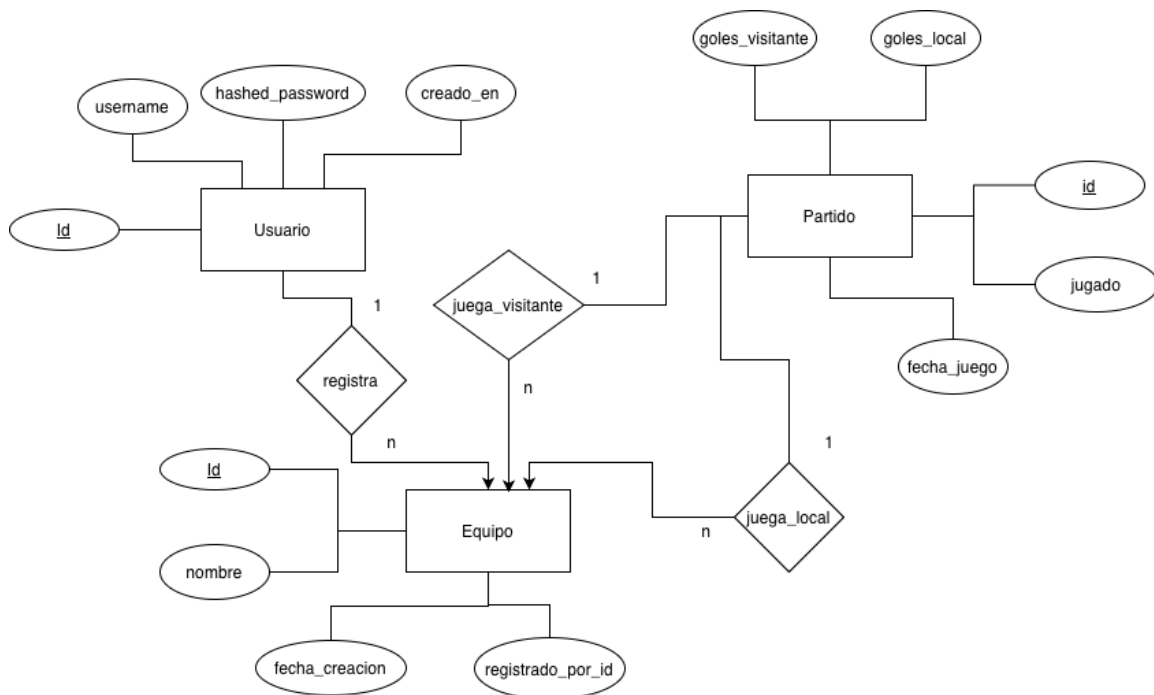


Figura 1: Diagrama Entidad-Relación de la base de datos.

10.2. Modelo Relacional

El modelo relacional implementado en PostgreSQL se muestra en la Figura 2.

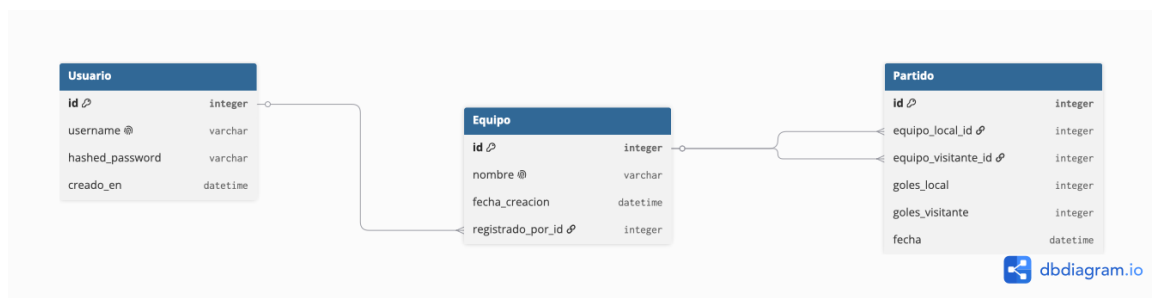


Figura 2: Modelo relacional implementado en PostgreSQL.

11. Diccionario de Datos

11.1. Entidad Usuario

Campo	Tipo	Restricción	Descripción
id	Integer	PK	Identificador único del usuario.
username	VARCHAR(50)	UNIQUE	Nombre utilizado para autenticación.
hashed_password	VARCHAR(255)	NOT NULL	Contraseña cifrada mediante bcrypt.
creado_en	DATETIME	NOT NULL	Fecha de creación del registro.

11.2. Entidad Equipo

Campo	Tipo	Restricción	Descripción
id	Integer	PK	Identificador del equipo.
nombre	VARCHAR(100)	UNIQUE	Nombre oficial del equipo.
fecha_creacion	DATETIME	NOT NULL	Fecha de registro del equipo.
registrado_por_id	Integer	FK	Usuario que creó el equipo.

11.3. Entidad Partido

Campo	Tipo	Restricción	Descripción
id	Integer	PK	Identificador único del partido.
equipo_local_id	Integer	FK	Identificador del equipo local.
equipo_visitante_id	Integer	FK	Identificador del equipo visitante.
goles_local	Integer	Nullable	Cantidad de goles anotados por el equipo local.
goles_visitante	Integer	Nullable	Cantidad de goles anotados por el equipo visitante.
jugado	Boolean	NOT NULL	Indica si el partido ya tiene un marcador registrado.
fecha_juego	DATETIME	Nullable	Fecha programada o fecha en la que se disputó el encuentro.

12. Reglas de Negocio

La implementación considera las siguientes reglas:

1. El sistema únicamente permite registrar un máximo de cuatro equipos.
2. Cada nombre de equipo debe ser único.

3. El fixture solamente puede generarse cuando existen cuatro equipos registrados.
4. Se generan automáticamente seis encuentros bajo el formato todos contra todos.
5. Los marcadores pueden modificarse en cualquier momento.
6. No se aceptan valores negativos para los goles registrados.
7. La clasificación se calcula utilizando:
 - Puntos obtenidos.
 - Diferencia de gol.
 - Goles a favor.

13. Implementación de la API REST

La solución expone todos sus servicios mediante una API REST desarrollada con FastAPI.

13.1. Autenticación

Método	Endpoint	Descripción
POST	/auth/login	Inicio de sesión y generación del JWT

13.2. Equipos

Método	Endpoint	Operación
POST	/equipos	Crear equipo
GET	/equipos	Listar equipos
GET	/equipos/{id}	Consultar equipo
PUT	/equipos/{id}	Actualizar equipo
DELETE	/equipos/{id}	Eliminar equipo

13.3. Partidos

Método	Endpoint	Operación
POST	/partidos/generar-fixture	Generar fixture
GET	/partidos	Consultar partidos
GET	/partidos/{id}	Obtener partido
PUT	/partidos/{id}/marcador	Registrar marcador

13.4. Tabla de posiciones

Método	Endpoint	Operación
GET	/tabla-posiciones	Obtener clasificación

13.5. Reinicio del torneo

Método	Endpoint	Operación
DELETE	/torneo/reiniciar	Reiniciar cuadrangular

14. Buenas Prácticas de Desarrollo

Durante el desarrollo del proyecto se aplicaron diversas buenas prácticas de ingeniería de software con el objetivo de obtener una solución mantenible, escalable y segura.

14.1. Arquitectura Modular

El código se organizó por capas, separando claramente las responsabilidades entre la exposición de la API, la lógica de negocio, la persistencia y la validación de datos.

Esta separación facilita el mantenimiento del proyecto y reduce el acoplamiento entre componentes.

14.2. Separación de Responsabilidades

Cada módulo cumple una función específica:

- **api**: recibe las solicitudes HTTP y devuelve las respuestas correspondientes.
- **services**: implementa las reglas de negocio del torneo.
- **models**: define las entidades persistentes mediante SQLAlchemy.
- **schemas**: valida la información de entrada y salida utilizando Pydantic.
- **db**: administra la conexión con PostgreSQL.

14.3. Validación de Datos

Todos los datos recibidos por la API son validados antes de ser procesados, evitando inconsistencias y errores en la base de datos.

Entre las validaciones implementadas destacan:

- Máximo de cuatro equipos registrados.
- Nombres de equipos únicos.
- Marcadores con valores enteros no negativos.
- Restricción para evitar generar múltiples veces el fixture.

14.4. Manejo de Excepciones

La API devuelve códigos HTTP semánticamente correctos:

- 200 OK para consultas exitosas.
- 201 Created para creación de recursos.
- 204 No Content para eliminaciones exitosas.
- 400 Bad Request cuando se incumplen reglas de negocio.
- 401 Unauthorized cuando no existe autenticación válida.
- 404 Not Found cuando el recurso solicitado no existe.

15. Autenticación y Seguridad

La aplicación implementa autenticación basada en JSON Web Tokens (JWT).

El proceso de autenticación consiste en:

1. El usuario envía sus credenciales al endpoint de inicio de sesión.
2. Si las credenciales son válidas, el servidor genera un token JWT firmado.
3. El cliente envía dicho token en el encabezado:

Authorization: Bearer <token>

4. Los endpoints protegidos verifican la autenticidad del token antes de ejecutar cualquier operación.

Para facilitar la evaluación del proyecto se configuró un usuario administrador inicial.

Usuario	admin
Contraseña	Admin2026

16. Persistencia de Datos

La persistencia del sistema se implementó utilizando PostgreSQL como gestor de bases de datos relacional.

El acceso a los datos se realiza mediante SQLAlchemy 2.0 como ORM, mientras que Alembic administra las migraciones del esquema.

Esta aproximación permite desacoplar la lógica de negocio del lenguaje SQL y facilita la evolución futura del modelo de datos.

17. Contenedorización con Docker

Con el objetivo de simplificar la instalación y despliegue, la aplicación fue empaquetada mediante Docker.

El archivo `docker-compose.yml` orquesta automáticamente los servicios necesarios para ejecutar el sistema.

La puesta en marcha puede realizarse mediante:

```
docker compose up --build
```

Una vez iniciado el entorno, se encuentran disponibles:

- API REST: <http://localhost:8000>
- Documentación Swagger: <http://localhost:8000/docs>
- Interfaz Web: <http://localhost:8000/app/>

18. Despliegue en Producción

La solución fue desplegada utilizando Microsoft Azure.

La infraestructura empleada comprende:

- Azure App Service para alojar la aplicación.
- Azure Database for PostgreSQL como base de datos administrada.
- Docker Hub como repositorio público de imágenes.

La aplicación puede accederse desde:

Interfaz Web

<https://futbol-api-milu2662.azurewebsites.net/app/>

Documentación Swagger

<https://futbol-api-milu2662.azurewebsites.net/docs>

19. Control de Versiones

Todo el desarrollo fue gestionado mediante Git utilizando GitHub como plataforma de alojamiento del repositorio.

Repositorio oficial:

<https://github.com/Milu2662/Futbol-api>

El proyecto emplea una estrategia basada en ramas de desarrollo y utiliza integración continua mediante GitHub Actions para ejecutar automáticamente las pruebas antes de integrar cambios.

20. Pruebas Automatizadas

Con el fin de garantizar la calidad del software, se desarrolló una batería de pruebas automatizadas utilizando Pytest.

Entre los escenarios cubiertos se encuentran:

- Inicio de sesión exitoso.
- Rechazo de credenciales inválidas.
- Creación de equipos.
- Restricción del máximo de cuatro equipos.
- Validación de nombres duplicados.
- Generación automática del fixture.
- Registro de marcadores.
- Rechazo de marcadores negativos.
- Cálculo correcto de la tabla de posiciones.
- Eliminación de equipos y borrado en cascada de partidos.

La ejecución de las pruebas puede realizarse mediante:

```
pytest -v
```

21. Documentación Automática

Gracias al uso de FastAPI, la documentación OpenAPI se genera automáticamente.

Esto permite visualizar y probar todos los endpoints desde una interfaz gráfica accesible en:

<https://futbol-api-milu2662.azurewebsites.net/docs>

La documentación incluye:

- Descripción de endpoints.
- Parámetros de entrada.
- Modelos de respuesta.
- Códigos HTTP.
- Autenticación mediante JWT.

22. Evidencias de Funcionamiento

22.1. Inicio de Sesión



The image shows a login screen for a system titled "CUADRANGULAR DE FÚTBOL". The screen has a light green background with a white rounded rectangle in the center. Inside the rectangle, the title "CUADRANGULAR DE FÚTBOL" is at the top in a dark green font. Below it, the text "Inicia sesión" is displayed in a large, bold, black font. There are two input fields: the first is labeled "Usuario" and the second is labeled "Contraseña", both in a dark gray font. Each label is positioned to the left of its corresponding light green rounded input field. At the bottom of the white rectangle is a dark green rounded button with the text "Ingresar" in white.

Figura 3: Pantalla de autenticación del sistema.

22.2. Administración de Equipos



Figura 4: Gestión de equipos registrados.

22.3. Gestión de Marcadores

CUADRANGULAR · TODOS CONTRA TODOS

Centro de control del torneo

Marcadores

Tabla de posiciones

Equipos

Marcadores

Los Leones
VS
Los Hurones

1 - 1

JUGADO

Guardar

Los Leones
VS
Las Águilas

2 - 3

JUGADO

Guardar

Los Leones
VS
Las Panteras

0 - 4

JUGADO

Guardar

Los Hurones
VS
Las Águilas

2 - 1

JUGADO

Guardar

Los Hurones
VS
Las Panteras

2 - 0

JUGADO

Guardar

Las Águilas
VS
Las Panteras

3 - 5

JUGADO

Guardar

Figura 5: Registro y modificación de marcadores.

19

22.4. Tabla de Posiciones

Cerrar sesión

CUADRANGULAR · TODOS CONTRA TODOS

Centro de control del torneo

Marcadores Tabla de posiciones Equipos

Tabla de posiciones

#	EQUIPO	PJ	G	E	P	GF	GC	DG	PTS
1	Los Hurones	3	2	1	0	5	2	+3	7
2	Las Panteras	3	2	0	1	9	5	+4	6
3	Las Águilas	3	1	0	2	7	9	-2	3
4	Los Leones	3	0	1	2	3	8	-5	1

Figura 6: Tabla de posiciones calculada dinámicamente.

22.5. Documentación Swagger

Cuadrangular Fútbol API 1.0.2 OAS 3.1
[/openapi.json](#)
API REST para gestionar un cuadrangular de fútbol: equipos, partidos y tabla de posiciones.

Authorize

Autenticación

POST /auth/login Login

Equipos

GET /equipos/ Listar Equipos

POST /equipos/ Crear Equipo

GET /equipos/{equipo_id} Obtener Equipo

PUT /equipos/{equipo_id} Actualizar Equipo

DELETE /equipos/{equipo_id} Eliminar Equipo

Figura 7: Documentación interactiva de la API - Swagger.

22.6. Repositorio GitHub

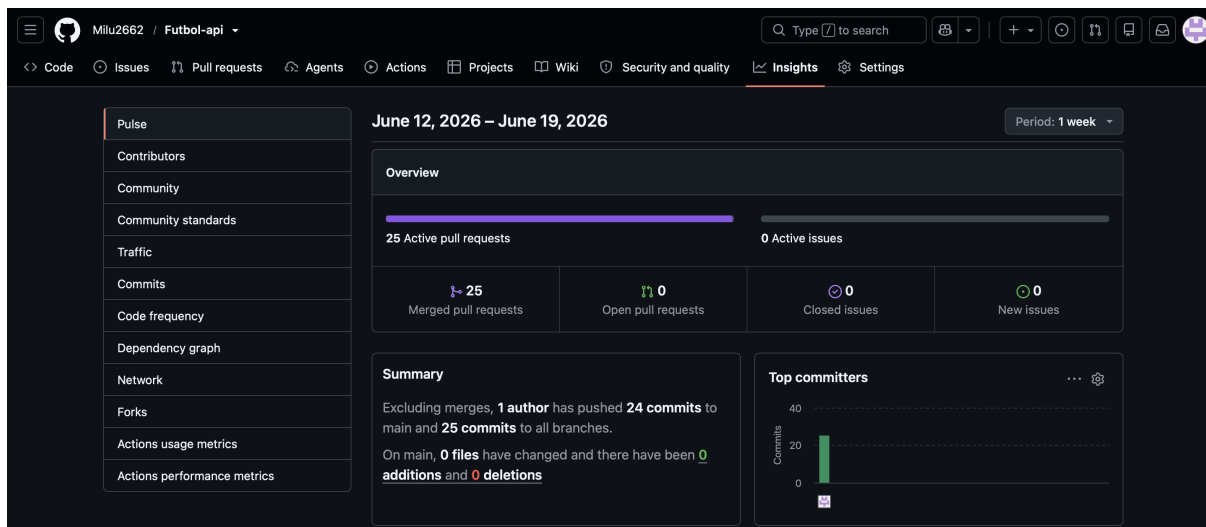


Figura 8: Repositorio del proyecto alojado en GitHub

23. Conclusiones

El proyecto desarrollado cumple satisfactoriamente con todos los requerimientos planteados en el enunciado de la prueba técnica. La solución implementa una API REST robusta para la gestión de un cuadrangular de fútbol, permitiendo registrar equipos, generar automáticamente el fixture bajo el formato todos contra todos, administrar los marcadores y calcular la tabla de posiciones en tiempo real.

Desde el punto de vista técnico, se empleó una arquitectura modular basada en FastAPI y SQLAlchemy, con persistencia en PostgreSQL, autenticación mediante JWT y documentación automática utilizando Swagger. Adicionalmente, se desarrolló una interfaz web funcional que facilita la interacción con el sistema y mejora la experiencia del usuario.

El proyecto incorpora prácticas modernas de desarrollo como el uso de Docker para la contenedorización, GitHub para el control de versiones, GitHub Actions para integración continua y Microsoft Azure para el despliegue en producción, lo que evidencia una solución alineada con estándares actuales de ingeniería de software.

Finalmente, la implementación de pruebas automatizadas, el manejo adecuado de excepciones, la validación de reglas de negocio y la clara separación entre capas garantizan un producto confiable, mantenible y preparado para futuras ampliaciones.